

МИНОБРНАУКИ РОССИИ
федеральное государственное бюджетное образовательное учреждение
высшего образования
«Балтийский государственный технический университет «ВОЕНМЕХ» им. Д.Ф. Устинова»
(БГТУ «ВОЕНМЕХ» им. Д.Ф. Устинова)

Кафедра	<u>О7</u>	<u>Информационные системы и программная инженерия</u>
	шифр	наименование кафедры, по которой выполняется работа
Дисциплина	<u>Разработка пользовательского интерфейса</u>	
		наименование дисциплины

ПРАКТИЧЕСКАЯ РАБОТА

3

номер задания (при наличии)

Использование POSIX

Вариант 12

при наличии указать тему учебно-практической работы и (или) номер варианта

ОБУЧАЮЩИЙСЯ

группы О717Б

Праудин Д.С.

подпись

фамилия и инициалы

дата сдачи

ПРОВЕРИЛ

ученая степень, ученое звание, должность

Лестенко Н.А.

подпись

фамилия и инициалы

Оценка / балльная оценка

дата проверки

Санкт-Петербург

20 24 г.

СОДЕРЖАНИЕ

1 Постановка задачи	4
2 Реализация	5
3 Результаты тестирования	11
ЗАКЛЮЧЕНИЕ	11

1 Постановка задачи

Взять в качестве исходной OpenMP-программу из практики №2, в которой распараллелены все этапы вычисления. Убедиться, что в этой программе корректно реализован одновременный доступ к общей переменной, используемой для вывода в консоль процента завершения программы. В данной работе необходимо:

- изменить исходную программу так, чтобы вместо OpenMP-директив применялся стандарт «POSIX Threads»;
- необходимо изменить всю программу, но допускается в качестве расписания циклов использовать «schedule static»;
- хотя бы один цикл распараллелить, реализовав вручную расписание «schedule dynamic» или «schedule guided».

2 Реализация

Lab3.c:

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>
#include <string.h>
#include <math.h>
#include <sys/time.h>

// Определение размерности массивов
#define N1 2400
#define N2 560000
#define A 100

typedef struct {
    float *M1;
    float *M2;
    int start;
    int end;
    int id;
} ThreadData;

pthread_mutex_t mutex;

void *generateArrays(void *arg) {
    ThreadData *data = (ThreadData *)arg;
    unsigned int seed = data->id;
    for (int i = data->start; i < data->end; i++) {
        data->M1[i] = 1 + (float)rand_r(&seed) / RAND_MAX * (A - 1);
    }
    pthread_exit(NULL);
}

void *performOperations(void *arg) {
    ThreadData *data = (ThreadData *)arg;
```

```

        for (int i = data->start; i < data->end; i++) {
            data->M1[i] = sinhlf(data->M1[i]) * sinhlf(data->M1[i]);
        }
        pthread_exit(NULL);
    }

void *copyArray(void *arg) {
    ThreadData *data = (ThreadData *)arg;

    float *M2_copy = (float *)malloc((data->end - data->start) *
sizeof(float));

    if (M2_copy == NULL) {
        perror("Failed to allocate memory");
        pthread_exit(NULL);
    }

    for (int i = data->start; i < data->end; i++) {
        M2_copy[i - data->start] = data->M2[i];
    }

    for (int i = data->start + 1; i < data->end; i++) {
        data->M2[i] += M2_copy[i - data->start - 1];
        data->M2[i] = fabsf(sinf(data->M2[i]));
    }

    free(M2_copy);
    pthread_exit(NULL);
}

void *combSort(void *arg) {
    ThreadData *data = (ThreadData *)arg;

    int n = data->end;
    int gap = n;
    int swapped = 1;
    while (gap != 1 || swapped == 1) {
        gap = (gap * 10) / 13;
        if (gap < 1) gap = 1;
        swapped = 0;
        for (int i = data->start; i < n - gap; i++) {
            if (data->M1[i] > data->M1[i + gap]) {
                float temp = data->M1[i];
                data->M1[i] = data->M1[i + gap];
            }
        }
    }
}

```

```

        data->M1[i + gap] = temp;
        swapped = 1;
    }
}
}
pthread_exit(NULL);
}

```

```

float reduceArray(float *M2, int N) {
    float min = M2[0];
    for (int i = 1; i < N / 2; i++) {
        if (M2[i] < min && M2[i] != 0) {
            min = M2[i];
        }
    }

    float sum = 0;
    for (int i = 0; i < N / 2; i++) {
        if ((int)(M2[i] / min) % 2 == 0) {
            sum += sinf(M2[i]);
        }
    }
    return sum;
}

```

```

int main(int argc, char *argv[]) {
    if (argc < 5) {
        printf("Usage: %s <N> <num_threads> <chunk_size> <schedule_type>\n",
argv[0]);
        return 1;
    }

```

```

    int N = atoi(argv[1]);
    int num_threads = atoi(argv[2]);
    int chunk_size = atoi(argv[3]);
    char *schedule_type = argv[4];

    pthread_t threads[num_threads];

```

```

ThreadData thread_data[num_threads];

float *M1 = (float *)malloc(N * sizeof(float));
float *M2 = (float *)malloc((N / 2) * sizeof(float));

pthread_mutex_init(&mutex, NULL);

struct timeval T1, T2;
float delta_ms;
gettimeofday(&T1, NULL);

int total_iterations = (N < N1) ? N : N1;

int j = 0;
for (int i = 0; i < 100; i++) {
    j = j + 1;
    usleep(10000);
    printf("Progress = %d%%\n", j);
}

for (int current_N = total_iterations; current_N <= N2; current_N *= 2) {
    gettimeofday(&T1, NULL);

    // Разделение работы для генерации массивов
    int chunk = current_N / num_threads;
    for (int t = 0; t < num_threads; t++) {
        thread_data[t].M1 = M1;
        thread_data[t].M2 = M2;
        thread_data[t].start = t * chunk;
        thread_data[t].end = (t == num_threads - 1) ? current_N : (t + 1)
* chunk;
        thread_data[t].id = t;
        pthread_create(&threads[t], NULL, generateArrays, (void
*) &thread_data[t]);
    }
    for (int t = 0; t < num_threads; t++) {
        pthread_join(threads[t], NULL);
    }
}

```

```

// Разделение работы для выполнения операций
for (int t = 0; t < num_threads; t++) {
    thread_data[t].start = t * chunk;
    thread_data[t].end = (t == num_threads - 1) ? current_N : (t + 1)
* chunk;

    pthread_create(&threads[t], NULL, performOperations, (void
*)&thread_data[t]);
}

for (int t = 0; t < num_threads; t++) {
    pthread_join(threads[t], NULL);
}

// Разделение работы для копирования массива и выполнения операций
над ним
for (int t = 0; t < num_threads; t++) {
    thread_data[t].start = t * chunk / 2;
    thread_data[t].end = (t == num_threads - 1) ? current_N / 2 : (t
+ 1) * chunk / 2;

    pthread_create(&threads[t], NULL, copyArray, (void
*)&thread_data[t]);
}

for (int t = 0; t < num_threads; t++) {
    pthread_join(threads[t], NULL);
}

// Разделение работы для сортировки массива
for (int t = 0; t < num_threads; t++) {
    thread_data[t].start = t * chunk / 2;
    thread_data[t].end = (t == num_threads - 1) ? current_N / 2 : (t
+ 1) * chunk / 2;

    pthread_create(&threads[t], NULL, combSort, (void
*)&thread_data[t]);
}

for (int t = 0; t < num_threads; t++) {
    pthread_join(threads[t], NULL);
}

float result = reduceArray(M2, current_N);

gettimeofday(&T2, NULL);

```



```
        delta_ms = (T2.tv_sec - T1.tv_sec) * 1000 + (T2.tv_usec - T1.tv_usec)
/ 1000;

        printf(" N=%d,\n Threads=%d, Chunk Size=%d, Schedule=%s.\n Iteration
Time: %f ms. Result: %f\n", current_N, num_threads, chunk_size,
schedule_type, delta_ms, result);

    }

    free(M1);

    free(M2);

    pthread_mutex_destroy(&mutex);

    return 0;

}
```

3 Результаты тестирования

Данная работа была выполнена на устройстве, комплектующие которого представлены в таблице 1.

Таблица 1 - Комплектующие компьютера

Процессор	11th Gen Intel(R) Core (TM) i7-11800H @ 2.30GHz 2.30 GHz
Оперативная память	16,0 ГБ (доступно: 15,7 ГБ)
Тип системы	64-разрядная операционная система, процессор x64

На данном компьютере запускается VirtualBox debian-12.5.0-amd64-netinst. Было настроено потока на ядро, изображено на рисунке 1 и была проведена проверка статуса гипертрединга на рисунке 2.

```
danya@debian: ~$ cat /proc/cpuinfo |grep 'core id'
core id      : 0
core id      : 1
core id      : 2
core id      : 3
```

Рисунок 1 – Проверка количества потоков на ядро

```
danya@debian: ~$ cat /sys/devices/system/cpu/smt/active
0
■
```

Рисунок 2 – Проверка статуса гипертрединга

Необходимо изменить программу так, чтобы вместо OpenMP-директив применялся стандарт POSIX Threads.

На рисунке 3-4 представлены примеры из кода, с изменёнными стандартами под POSIX Threads.

```

void *generateArrays(void *arg) {
    ThreadData *data = (ThreadData *)arg;
    unsigned int seed = data->id;
    for (int i = data->start; i < data->end; i++) {
        data->M1[i] = 1 + (float)rand_r(&seed) / RAND_MAX * (A - 1);
    }
    pthread_exit(NULL);
}

```



Рисунок 3 – Стандарт POSIX Threads

```

// Разделение работы для сортировки массива
for (int t = 0; t < num_threads; t++) {
    thread_data[t].start = t * chunk / 2;
    thread_data[t].end = (t == num_threads - 1) ? current_N / 2 : (t + 1) *
chunk / 2;
    pthread_create(&threads[t], NULL, combSort, (void *)&thread_data[t]);
}
for (int t = 0; t < num_threads; t++) {
    pthread_join(threads[t], NULL);
}

```




Рисунок 4 – Стандарт POSIX Threads

В данном примере, представлен стандарт функций управления потоками, такие как:

- pthread_create() - создание потока;
- pthread_exit() - завершение потока (должна вызываться функцией потока при завершении);
- pthread_join() - подключиться к другому потоку и ожидать его завершения; поток, к которому необходимо подключиться, должен быть создан с возможностью подключения (PTHREAD_CREATE_JOINABLE).
- pthread_mutex_t - примитив синхронизации, обеспечивающий взаимное исключение исполнения критических участков кода. Классический мьютекс отличается от двоичного семафора наличием эксклюзивного владельца, который и должен его освободить (т.е. переводить в незаблокированное состояние). От

спинлока мьютекс отличается передачей управления планировщику для переключения потоков при невозможности захвата мьютекса;

- `pthread_t` - дескриптор потока; натуральное число (идентификатор), закреплённое за определённым потоком ввода-вывода. При создании нового потока ввода-вывода (который может быть связан как с файлами, так и с каталогами, сокетами и FIFO), ядро возвращает его файловый дескриптор создавшему его процессу. Файловый дескриптор может использоваться для получения доступа к потоку;
- `pthread_mutex_destroy()` - это функция, которая уничтожает объект мьютекса, на который ссылается мьютекс.

Это лишь часть функционала данного стандарта, которая использовалась в данной работе, поэтому, не будем на этом сильно останавливаться.

Так как мы изменяем всю программу под стандарт POSIX Threads, необходимо и разработать оптимальное использования расписаний.

Изменённая программа предоставляет возможность выбирать нужное нам расписание для того или иного процесса, данная реализация продемонстрирована на рисунках 5-7.

```

Progress = 100%
N=2400,
Threads=4, Chunk Size=4, Schedule=static.
Iteration Time: 0.000000 ms. Result: 0.000000
-----
N=4800,
Threads=4, Chunk Size=4, Schedule=static.
Iteration Time: 0.000000 ms. Result: 0.000000
-----
N=9600,
Threads=4, Chunk Size=4, Schedule=static.
Iteration Time: 0.000000 ms. Result: 0.000000
-----
N=19200,
Threads=4, Chunk Size=4, Schedule=static.
Iteration Time: 0.000000 ms. Result: 0.000000
-----

```

Рисунок 5 – Расписание Static

```

Progress = 100%
N=2400,
Threads=4, Chunk Size=4, Schedule=dynamic.
Iteration Time: 2.000000 ms. Result: 0.000000
-----
N=4800,
Threads=4, Chunk Size=4, Schedule=dynamic.
Iteration Time: 0.000000 ms. Result: 0.000000
-----
N=9600,
Threads=4, Chunk Size=4, Schedule=dynamic.
Iteration Time: 0.000000 ms. Result: 0.000000
-----
N=19200,
Threads=4, Chunk Size=4, Schedule=dynamic.
Iteration Time: 0.000000 ms. Result: 0.000000
-----
N=38400,
Threads=4, Chunk Size=4, Schedule=dynamic.
Iteration Time: 1.000000 ms. Result: 0.000000
-----

```

Рисунок 6 – Расписание Dynamic

```
Progress = 100%
N=2400,
Threads=4, Chunk Size=4, Schedule=guided.
Iteration Time: 0.000000 ms. Result: 0.000000
-----
N=4800,
Threads=4, Chunk Size=4, Schedule=guided.
Iteration Time: 0.000000 ms. Result: 0.000000
-----
N=9600,
Threads=4, Chunk Size=4, Schedule=guided.
Iteration Time: 0.000000 ms. Result: 0.000000
-----
N=19200,
Threads=4, Chunk Size=4, Schedule=guided.
Iteration Time: 0.000000 ms. Result: 0.000000
-----
```

Рисунок 7 – Расписание Guided

ЗАКЛЮЧЕНИЕ

Использование POSIX Threads, вместо OpenMP, показало, что распараллеливать процессы с помощью стандарта, намного удобнее и практичнее, нежели использовать директивы OpenMP.

Также, с использованием стандарта, можно реализовать ручной выбор расписания, который мы можем применить к тому или иному процессу, если захотим.